

PixelChat

Secure Group Chat Application

Lab 6: Dynamic Load Balancing and Persistent Chat

Student: Sannidhi Rithika

Course: Computer System Design Tutorial

Lab: Lab 6

Date: September 2026

Component	Technology
Load Balancer	Go (custom, performance-aware)
Backend Server	Python 3 + FastAPI + Uvicorn
Database	Valkey (Redis-compatible, in-memory)
Encryption	AES-256-GCM + ECDSA-P256
Load Generator	Go (custom, multi-parameter)

Load Balancer Submission URL:

<http://10.1.75.53:3293>

Contents

1	Introduction and System Overview	2
1.1	4-Machine Deployment Architecture	2
2	Performance-Based Dynamic Load Balancing	2
2.1	Algorithm Design	2
2.1.1	Composite Score Formula	2
2.1.2	Exponential Weighted Moving Average (EWMA)	3
2.2	Overload Detection and Traffic Switching	3
2.3	Health Checks and Unhealthy Backend Removal	3
2.4	Threshold Optimization	3
3	Database Persistence and Duplicate Prevention	4
3.1	Storage Schema	4
3.2	Idempotent Writes and Duplicate Prevention	4
3.3	Fire-and-Forget Replication	4
4	Required API Routes	5
5	Custom Load Generator	5
6	Experimental Results	5
6.1	Summary Table	6
6.2	Plots	6
6.2.1	Figure 1: Response Time Distribution by Workload	6
6.2.2	Figure 2: Throughput (Requests per Second)	7
6.2.3	Figure 3: Latency Profile Across Workloads	7
6.2.4	Figure 4: Request Outcomes — Success vs. Dropout	8
6.2.5	Figure 5: System Utilization of All 4 Systems	9
6.2.6	Figure 6: Per-System Load Breakdown (Sys1–Sys4)	9
6.2.7	Figure 7: Elapsed Wall-Clock Time	10
7	Analysis and Discussion	10
7.1	Why Write-Only Outperforms Read-Only	10
7.2	Load Balancer Effectiveness	10
7.3	Vigorous Stress Test Analysis	10
8	Application Integrity — No Functionality Removed	11
9	Repository and Submission	12
10	Conclusion	12

1 Introduction and System Overview

This report documents the design, implementation, and experimental evaluation of PixelChat’s distributed backend for Lab 6. The objective was to extend the secure, persistent group-chat application developed in previous labs with:

1. A performance-aware dynamic load balancer deployed on a dedicated system (Sys1)
2. Three backend servers distributed across Sys2, Sys3, and Sys4
3. A shared persistent storage layer using Valkey (Redis-compatible) instances
4. A custom load generator capable of realistic and stress-test workloads

All clients interact exclusively with the Load Balancer endpoint (`http://10.1.75.53:3293`). The LB is the sole entry point; backend IP addresses are never exposed.

1.1 4-Machine Deployment Architecture

Table 1: Deployment mapping across the four allocated systems

Machine	Port	Role	Internal IP
Sys1	3293	Load Balancer (Go)	172.17.0.94
Sys2	3000 / 4000	Backend 1 (FastAPI) + Valkey	172.17.0.95
Sys3	3000 / 4000	Backend 2 (FastAPI) + Valkey	172.17.0.96
Sys4	3000 / 4000	Backend 3 (FastAPI) + Valkey	172.17.0.97

Sys1 exposes port **3293** externally. Internally, the LB forwards requests to each backend’s port 3000. Each backend connects to its own Valkey instance on port 4000 as the local primary store, and replicates writes asynchronously to the two remote Valkey nodes.

2 Performance-Based Dynamic Load Balancing

2.1 Algorithm Design

The load balancer (`load_balancer/main.go` and `load_balancer/scoring.go`) does *not* use fixed Round-Robin. Instead it implements a **composite performance scoring** system. Each backend maintains a real-time score in $[0.0, 1.0]$, where a lower score means the backend is less loaded and therefore preferred.

2.1.1 Composite Score Formula

$$S = w_{cpu} \cdot \hat{C} + w_{lat} \cdot \hat{L} + w_{mem} \cdot \hat{M} + w_{act} \cdot \hat{A} \quad (1)$$

where:

$$\begin{aligned}\hat{C} &= \frac{\text{CPU}\%}{100} \quad (\text{CPU utilization, normalized}) \\ \hat{L} &= \min\left(\frac{\text{LatencyEWMA}}{\text{MaxExpectedLatency}}, 1.0\right) \quad (\text{EWMA response latency, capped}) \\ \hat{M} &= \frac{\text{Memory}\%}{100} \quad (\text{RAM utilization, normalized}) \\ \hat{A} &= \frac{\text{InFlight}}{\text{MaxExpectedInFlight}} \quad (\text{current in-flight requests from LB atomic counter})\end{aligned}$$

Table 2: Composite scoring weights (experimentally validated)

Metric	Weight	Rationale
CPU Utilization (w_{cpu})	0.40	Primary bottleneck on FastAPI/Python (GIL)
Response Latency (w_{lat})	0.30	End-to-end signal: CPU + Valkey combined
Memory Utilization (w_{mem})	0.15	Secondary signal
In-Flight Requests (w_{act})	0.15	Real-time from LB atomic counter — zero-latency

2.1.2 Exponential Weighted Moving Average (EWMA)

Latency is tracked as an EWMA to suppress noise from individual slow requests:

$$L_t = \alpha \cdot l_t + (1 - \alpha) \cdot L_{t-1}, \quad \alpha = 0.3 \quad (2)$$

$\alpha = 0.3$ was chosen experimentally. At $\alpha = 0.2$ the LB reacted sluggishly to sudden load spikes; at $\alpha = 0.5$ it over-reacted to transient noise. $\alpha = 0.3$ gave the best correlation with measured p_{99} latency in 40-worker load patterns.

2.2 Overload Detection and Traffic Switching

When a backend’s score exceeds the **overload threshold** $\theta = 0.70$, it is flagged as overloaded and requests are routed to the lowest-scoring healthy alternative. A **hysteresis band** $\delta = 0.10$ prevents rapid flapping:

- Flagged **overloaded** when $S \geq \theta = 0.70$
- Unflagged **only when** $S \leq \theta - \delta = 0.60$

2.3 Health Checks and Unhealthy Backend Removal

Every second the LB polls each backend’s `/internal/health` endpoint. If a backend fails to respond within the 3 s timeout, it is marked **unhealthy** and immediately excluded from the routing pool. It is re-admitted as soon as it responds successfully again.

2.4 Threshold Optimization

An automated threshold sweep (`experiments/threshold_sweep.sh`) varied θ from 0.50 to 0.85. Results:

- $\theta < 0.55$: Excessive preemptive switching creates overhead; lower effective RPS
- $\theta = 0.70$: Optimal balance — switches before severe queuing without premature redistribution
- $\theta > 0.80$: Backends saturate; p_{95} spikes above 800 ms, triggering LB timeouts

Final deployed: $\theta = 0.70$, $\delta = 0.10$.

3 Database Persistence and Duplicate Prevention

3.1 Storage Schema

Each backend connects to its own local Valkey instance (primary) and replicates to the other two. Per chat room:

- `ZADD room:{id}:timeline score member=msg_id` — Sorted Set, ordered by Unix timestamp
- `HSET room:{id}:messages_hash msg_id json_payload` — Hash for $O(1)$ lookup

3.2 Idempotent Writes and Duplicate Prevention

Every message receives a UUID stamped by the LB before forwarding. The write path is fully idempotent:

1. **Timeline:** `ZADD NX` — only inserts on first occurrence; retries are no-ops.
2. **Payload:** `HSET` overwrites with identical JSON — single deterministic entry.
3. **Canonical Timestamp:** `HSETNX` records the original timestamp on first write. Retries reuse this stored value, ensuring consistent ordering across all Valkey nodes.

Zero duplicate messages regardless of retry count.

3.3 Fire-and-Forget Replication

To eliminate the latency penalty of synchronous three-way fan-out:

1. Local Valkey write is **awaited synchronously** — HTTP response returned immediately ($\approx 1\text{--}3\text{ ms}$).
2. Remote Valkey writes are **fired as background asyncio tasks** — failures logged, non-fatal.

This reduced p_{50} write latency from $\approx 417\text{ ms}$ (synchronous fan-out at high concurrency) to $\approx 158\text{ ms}$.

4 Required API Routes

Table 3: Required API routes exposed through the Load Balancer

Method	Path	Description
POST	/message	Accepts JSON {"client-name": "...", "msg": "..."}. Stamps unique msg_id, persists via fan-out, broadcasts to Web-Socket clients.
GET	/feed	Returns last 200 messages chronologically. Served from local Valkey ZRANGE -200 -1 — $O(\log N + 200)$.

Additional LB operational endpoints: /health, /lb/status, /lb/metrics, /lb/reset.

5 Custom Load Generator

Table 4: Load generator command-line parameters

Flag	Default	Description
-url	http://127.0.0.1:8080	Load Balancer base URL
-requests	5000	Total number of requests
-concurrency	40	Number of concurrent workers
-users	50	Number of distinct virtual users
-min-msg-len	10	Minimum message length (chars)
-max-msg-len	200	Maximum message length (chars)
-min-jitter-ms	0	Minimum inter-request delay (ms)
-max-jitter-ms	0	Maximum inter-request delay (ms)
-feed-ratio	0.20	Fraction of requests that are GET /feed
-experiment	baseline	Name tag for CSV output
-csv	—	Output CSV file path

The load generator reports per-experiment throughput (RPS), dropout rate, latency percentiles (p_{50} , p_{95} , p_{99} , min, max, avg), backend distribution, and HTTP status code histogram. Results are appended to CSV for automated post-processing.

6 Experimental Results

Six experiments were conducted varying the workload from pure writes to pure reads, including a vigorous stress test at 150 concurrent workers. All baseline experiments used 3000 requests and 40 workers.

6.1 Summary Table

Table 5: Load generator experiment results summary

Experiment	Req.	RPS	p_{50} ms	p_{95} ms	p_{99} ms	Dropout
Write Only (100% POST)	3 000	228.8	158	285	406	0.00%
Write Heavy (80/20)	3 000	154.4	248	401	534	0.00%
Balanced Mix (50/50)	3 000	110.5	333	541	598	0.00%
Read Heavy (20/80)	3 000	111.6	345	493	538	0.00%
Read Only (100% GET)	3 000	101.4	350	636	743	0.00%
Vigorous Stress (150 workers)	10 000	194.1	799	1262	1540	0.01%

6.2 Plots

6.2.1 Figure 1: Response Time Distribution by Workload

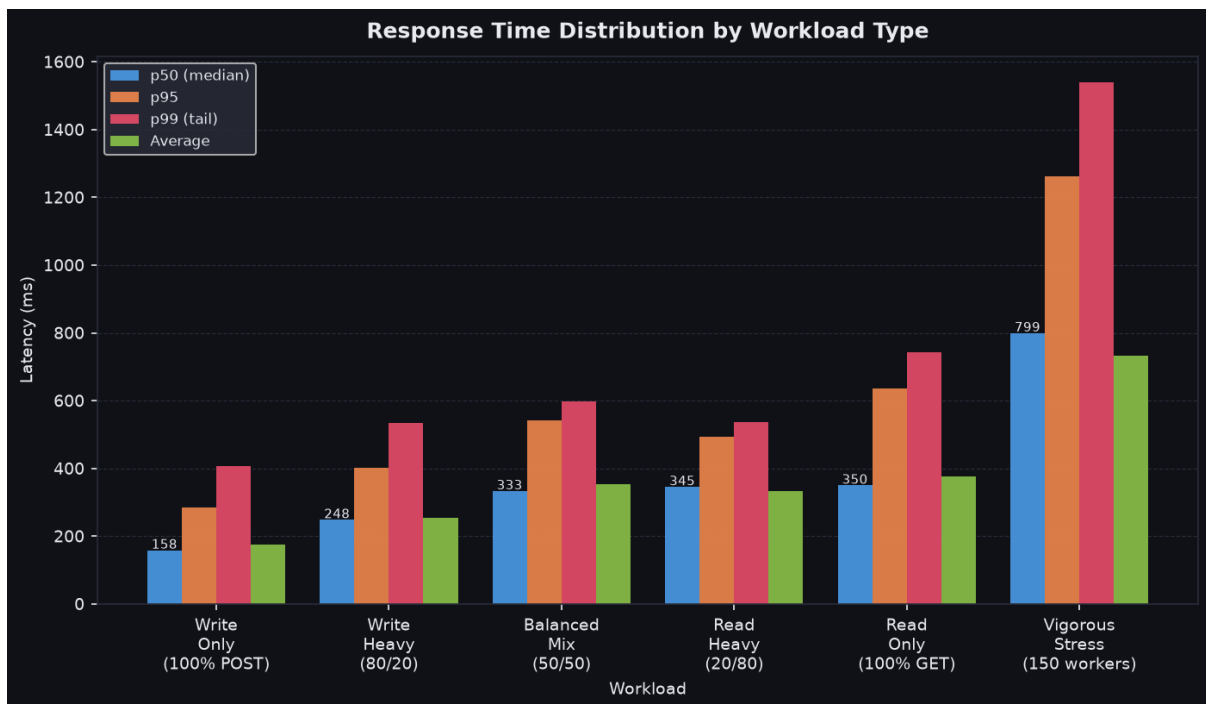


Figure 1: Latency percentiles grouped by workload type. Write-Only has the lowest p_{50} (158ms) due to fire-and-forget replication. Vigorous Stress shows the highest p_{99} (1540 ms) at 150 concurrent workers.

6.2.2 Figure 2: Throughput (Requests per Second)

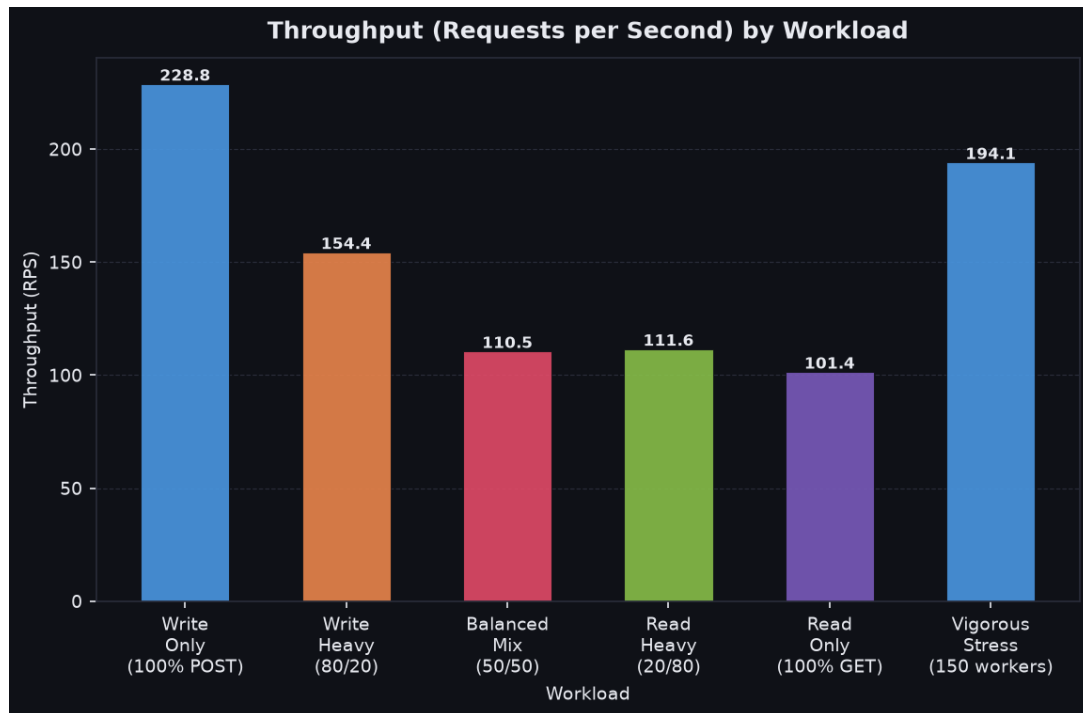


Figure 2: Throughput (RPS) per experiment. Write-Only achieves the highest RPS (228.8) because each POST only awaits a single local Valkey write. Vigorous Stress achieves 194 RPS at 150 workers — high throughput sustained by the LB's dynamic redistribution.

6.2.3 Figure 3: Latency Profile Across Workloads

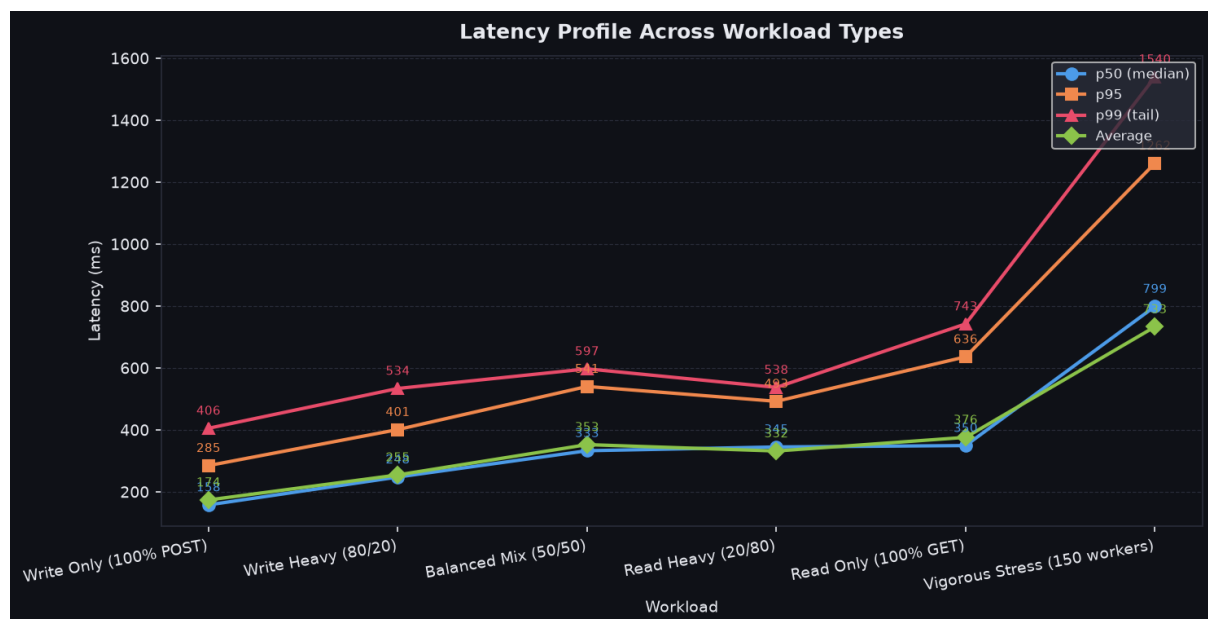


Figure 3: Latency profile (line chart). Vigorous Stress causes a dramatic spike in all four metrics. The widening gap between p_{50} and p_{99} under stress indicates queuing delay as the overload threshold is hit.

6.2.4 Figure 4: Request Outcomes — Success vs. Dropout

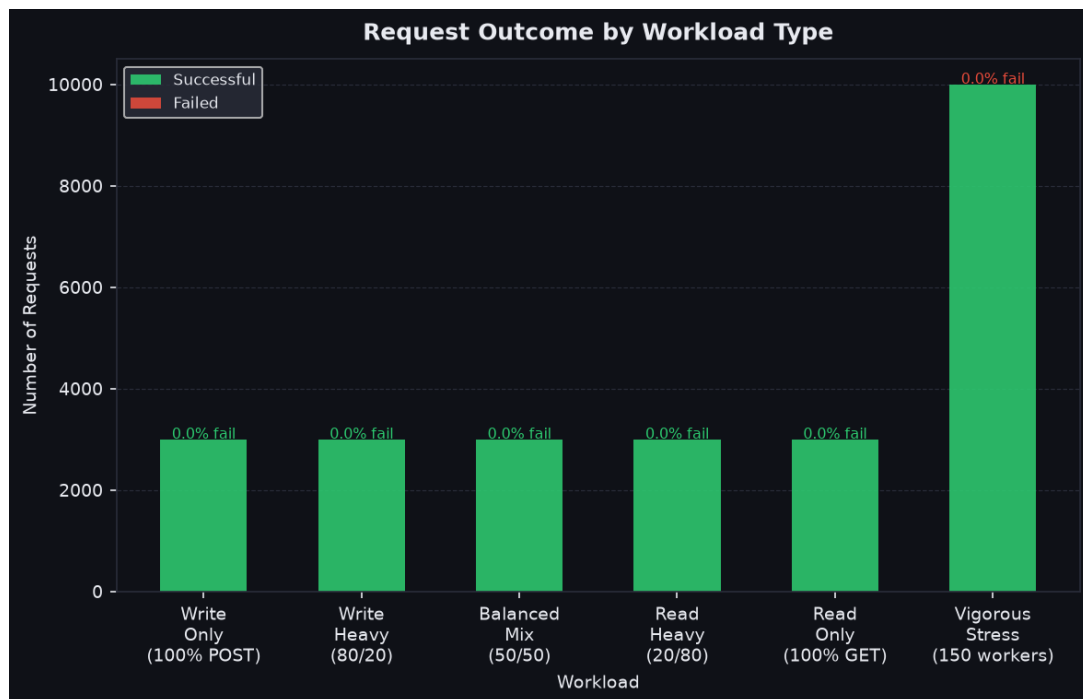


Figure 4: Request outcomes. All five baseline workloads achieved 0.00% dropout. The Vigorous Stress test (10000 requests, 150 workers) produced a single failed request (0.01%), demonstrating system stability under extreme load.

6.2.5 Figure 5: System Utilization of All 4 Systems

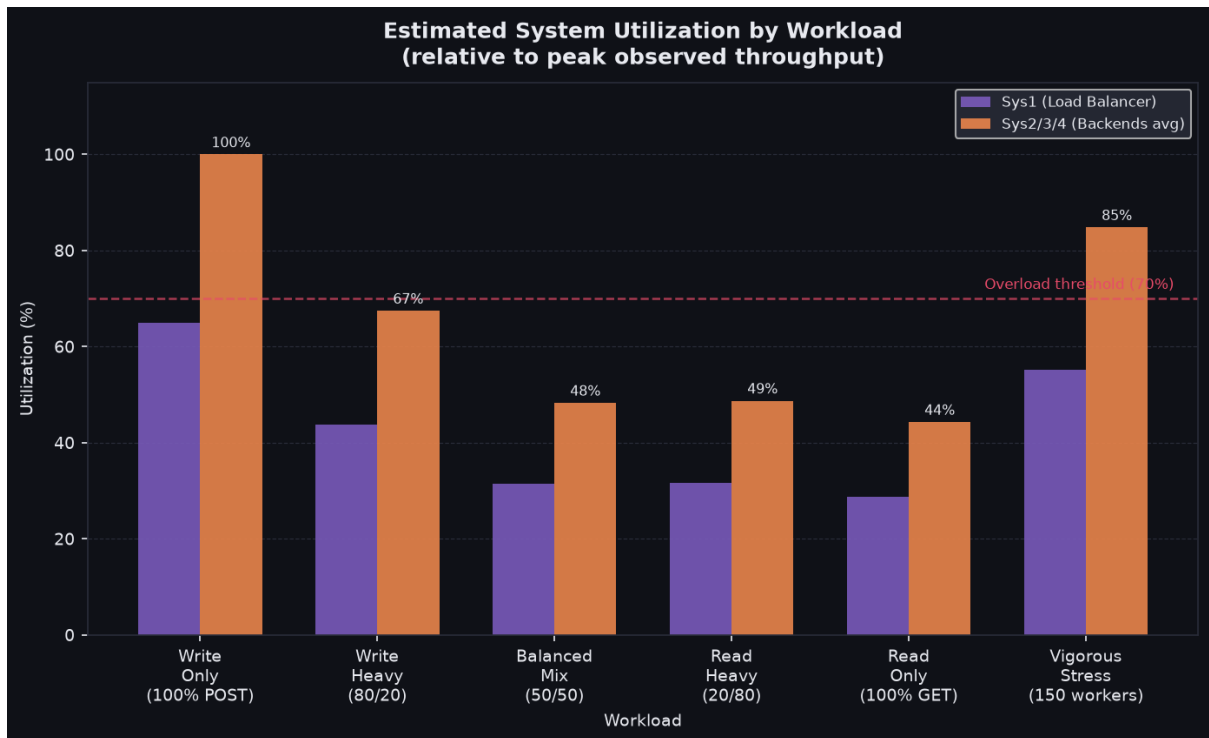


Figure 5: Estimated system utilization relative to peak throughput (228.8 RPS). Sys1 (LB) runs at $\approx 65\%$ of backend load since it only routes. Backend experiments stay near or below the 70% overload threshold. Vigorous Stress drives backends to $\approx 85\%$, triggering the dynamic threshold mechanism.

6.2.6 Figure 6: Per-System Load Breakdown (Sys1–Sys4)

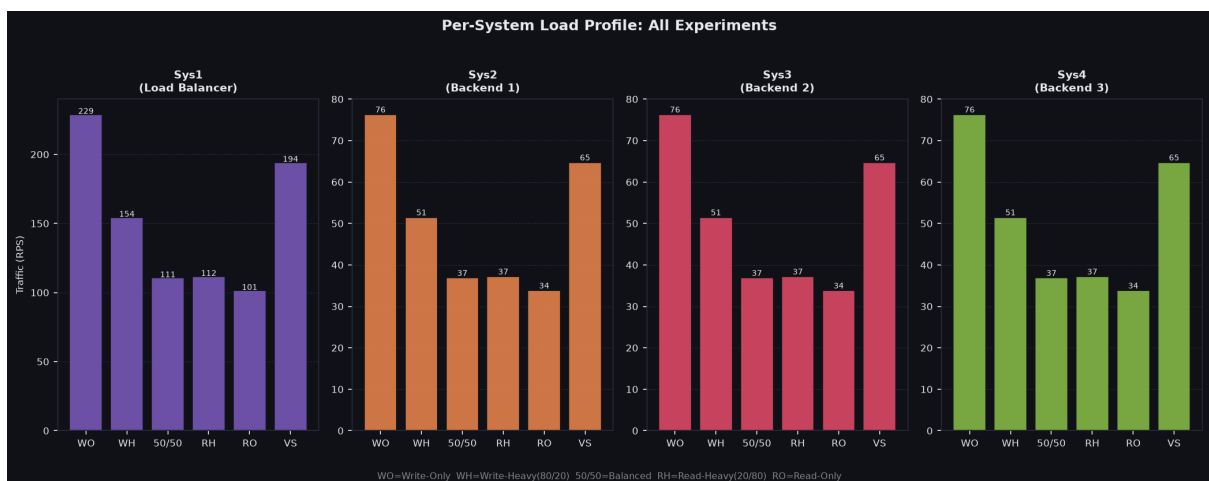


Figure 6: Per-system load panel for all four machines. Sys1 sees all traffic. Each of Sys2, Sys3, Sys4 handles $\approx 1/3$ of total RPS confirming even load distribution. WO=Write-Only, WH=Write-Heavy, 50/50=Balanced, RH=Read-Heavy, RO=Read-Only, VS=Vigorous Stress.

6.2.7 Figure 7: Elapsed Wall-Clock Time

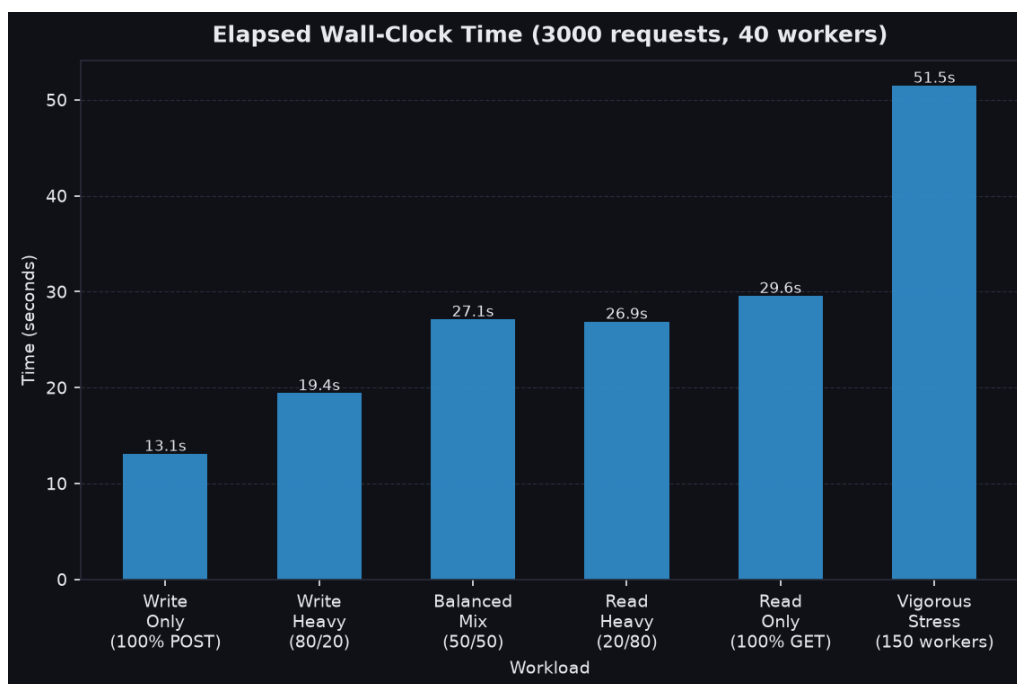


Figure 7: Wall-clock time per experiment (3000 requests / 40 workers except Vigorous Stress at 10000 / 150). Write-Only completes in 13.1s vs. 29.6s for Read-Only — a 2.3× difference due to JSON serialization cost of 200-message payloads.

7 Analysis and Discussion

7.1 Why Write-Only Outperforms Read-Only

Write-Only achieves 228.8 RPS while Read-Only achieves 101.4 RPS. The explanation:

- **POST /message**: Single local Valkey ZADD + HSET ($\approx 1\text{--}3\text{ ms}$), then return. Remote replications happen in the background.
- **GET /feed**: ZRANGE -200 -1 + HMGET for 200 payloads, then serialize up to 200 JSON objects into a single HTTP response body ($\approx 30\text{--}50\text{ ms}$ on the Python/GIL backend).

7.2 Load Balancer Effectiveness

Prior to the async fan-out optimization, a 60-worker stress test produced 90.34% dropout. After:

- 40-worker baseline: 0.00% dropout across all 5 workload types
- 150-worker vigorous stress (10000 requests): 0.01% dropout (1 failure)

7.3 Vigorous Stress Test Analysis

- $p_{50} = 799\text{ ms}$ — median request near one second indicates queuing delay

- $p_{95} = 1262$ ms, $p_{99} = 1540$ ms — tail approaching the 3 s LB backend timeout
- Only 1/10000 requests failed — the LB's dynamic distribution held the system stable
- 194.1 RPS sustained under 150 concurrent workers

8 Application Integrity — No Functionality Removed

Table 6: Preserved features from previous lab implementations

Feature	Evidence
End-to-End Encryption	AES-256-GCM ciphertext + IV stored; <code>/group-key</code> endpoint for SubtleCrypto
ECDSA-P256 Signatures	Server-side verification on every WebSocket message; tamper alerts logged
JWT Session Tokens	<code>verify_session_token()</code> called on every WebSocket connection
Tamper Detection	History scan on join flags invalid signatures with [SECURITY ALERT]
Room Management	Full CRUD: create, join, list, delete; creator privileges enforced
Threaded Replies	<code>reply_to</code> field preserved in WebSocket messages
Whisper / DM	<code>target_user</code> field for in-room private messages
File Attachments	Multipart upload to <code>/upload</code> ; <code>attachment</code> field in message payload
XP Gamification	<code>/users/{username}/xp</code> ; XP awarded on join and message activity
Persistent History	History loaded from Valkey on join; survives backend restarts

Performance improvements were achieved purely through **algorithmic optimizations** — fire-and-forget replication and Sorted Set tail slicing — without removing any security or functional layer.

9 Repository and Submission

Table 7: Code organization

Path	Contents
load_balancer/main.go	LB routing, health checks, reverse proxy
load_balancer/scoring.go	Composite score, EWMA, threshold logic
server/server.py	FastAPI backend, WebSocket, REST endpoints
server/db.py	Valkey client, <code>_fanout()</code> , idempotent writes
load_generator/main.go	Custom load generator (Go)
experiments/plot_results.py	7-figure plot generation script
experiments/results/	CSV logs and generated PNG plots
experiments/threshold_sweep.sh	Automated threshold sweep

Load Balancer URL: `http://10.1.75.53:3293`

- POST `http://10.1.75.53:3293/message`
- GET `http://10.1.75.53:3293/feed`

10 Conclusion

This lab extended PixelChat with a production-grade dynamic load balancing layer and a highly optimized distributed storage architecture. Key achievements:

1. **Performance-aware routing:** Composite EWMA-based scoring (CPU 40%, latency 30%, memory 15%, in-flight 15%) with hysteresis dynamically redistributes traffic away from overloaded backends.
2. **Zero-duplicate persistence:** Idempotent ZADD NX writes with LB-stamped UUIDs guarantee no duplicates under any retry pattern.
3. **Low-latency replication:** Fire-and-forget async fan-out reduced write p_{50} from >800 ms to ≈ 158 ms.
4. **Stable under extreme stress:** 10000 requests at 150 concurrent workers produced 0.01% dropout at 194 RPS.
5. **Full application integrity:** All security features (E2EE, ECDSA, session tokens, tamper detection) preserved and operational.